

Java异常处理基础

北京理工大学计算机学院
金旭亮



Java程序运行时异常的演示

```
static void exceptionCauseProgramTerminate() {  
    int i = 1, j = 0, k;  
    //如果不注释掉以下这句, 可以看到异常抛出, 程序中止  
    k = i / j;  
    System.out.println("result:" + k);  
}
```

AboutException.java



```
Run: AboutException x  
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\...  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at AboutException.exceptionCauseProgramTerminate(AboutException.java:13)  
    at AboutException.main(AboutException.java:5)  
  
Process finished with exit code 1
```

异常 (Exception) : 发生于程序执行期间, 表明出现了一个非法的运行状况。许多JDK中的方法在检测到非法情况时, 都会抛出一个异常对象, 如果不做处理, 将导致程序提前中止。

如何处理程序运行时出现的异常?

AboutException.java

Java中所有可捕获的异常都派生自Exception类。

```
try {  
    //可能发生运行错误的代码;  
}  
catch (异常类型 异常对象引用) {  
    //用于处理异常的代码  
}  
finally{  
    //用于“善后”的代码  
}
```

```
static void trycatchLetProgramContinue() {  
    int i = 1, j = 0, k;  
    //可以使用try...catch结构来捕获并处理异常,  
    //以避免程序提前中止  
    try {  
        k = i / j;    // 引发被零除异常  
    } catch (ArithmeticException e) {  
        System.out.println("被0除。" + e.getMessage());  
    } finally {  
        System.out.println("无论如何, 都要完成收尾工作");  
    }  
}
```

加上异常捕获之后, 程序可以顺利地执行完毕.....



```
Run: AboutException x  
"C:\Program Files\Java\jdk-15\bin\java.exe"  
被0除。 / by zero  
无论如何, 都要完成收尾工作  
Process finished with exit code 0
```

Java异常处理机制基本使用方法

```
try {  
    //可能发生运行错误的代码;  
}  
catch(异常类型1 异常对象引用 ) {  
    //用于处理异常1的代码  
}  
catch(异常类型2 异常对象引用) {  
    //用于处理异常2的代码  
}  
finally{  
    //用于“善后”的代码  
}
```

- 把可能会发生错误的代码放进try语句块中。
 - 当JVM检测到出现了一个错误时会抛出一个异常对象。异常处理代码会捕获并处理这个错误——**catch语句块中的代码用于处理错误。**
 - 当异常发生时，程序控制流程由try语句块跳转到与抛出异常相匹配的catch语句块。
 - 在通常情况下，如果try块被执行一次，则try块后只有一个catch块会被执行，绝不可能有多个catch块被执行。
-
- 不管是否有异常发生，finally语句块中的语句始终保证会被执行。
 - 如果没有找到合适的异常处理代码，JVM将会结束掉整个应用程序。

主动抛出异常

```
static void throwMyExceptions() {  
    try {  
        //必要时, 可以使用以下这句“主动抛出异常”  
        throw new Exception("某些不好的事情发生了.....");  
    } catch (Exception e) {  
        System.out.println(e.getMessage());  
    } finally {  
        System.out.println("无论如何, 都要完成收尾工作");  
    }  
}
```

AboutException.java

在实际开发中, 可以依据实际情况, 主动地中断当前执行的流程, 使用throw语句主动地抛出一个异常, 通知外界某个事情发生了.....

这种编程方式在实际开发中经常用到。

try语句块中的变量

try语句块中定义的变量，
catch块中无法访问！

```
static void variableInTry1() {  
    try {  
        int i = 100;  
        int j = 0;  
        System.out.println(i / j);  
    } catch (ArithmeticException e) {  
        //以下代码尝试访问变量i和j，是无法通过编译的  
        //System.out.println(i + "除以" + j + "出错！");  
    }  
}
```

```
static void variableInTry2() {  
    int i = 100;  
    int j = 0;  
    try {  
        System.out.println(i / j);  
    } catch (ArithmeticException e) {  
        //由于变量外置，现在可以访问得到i和j  
        System.out.println(i + "除以" + j + "出错！");  
    }  
}
```

必须将变量的定义移出try
块，catch块中才可以访问！

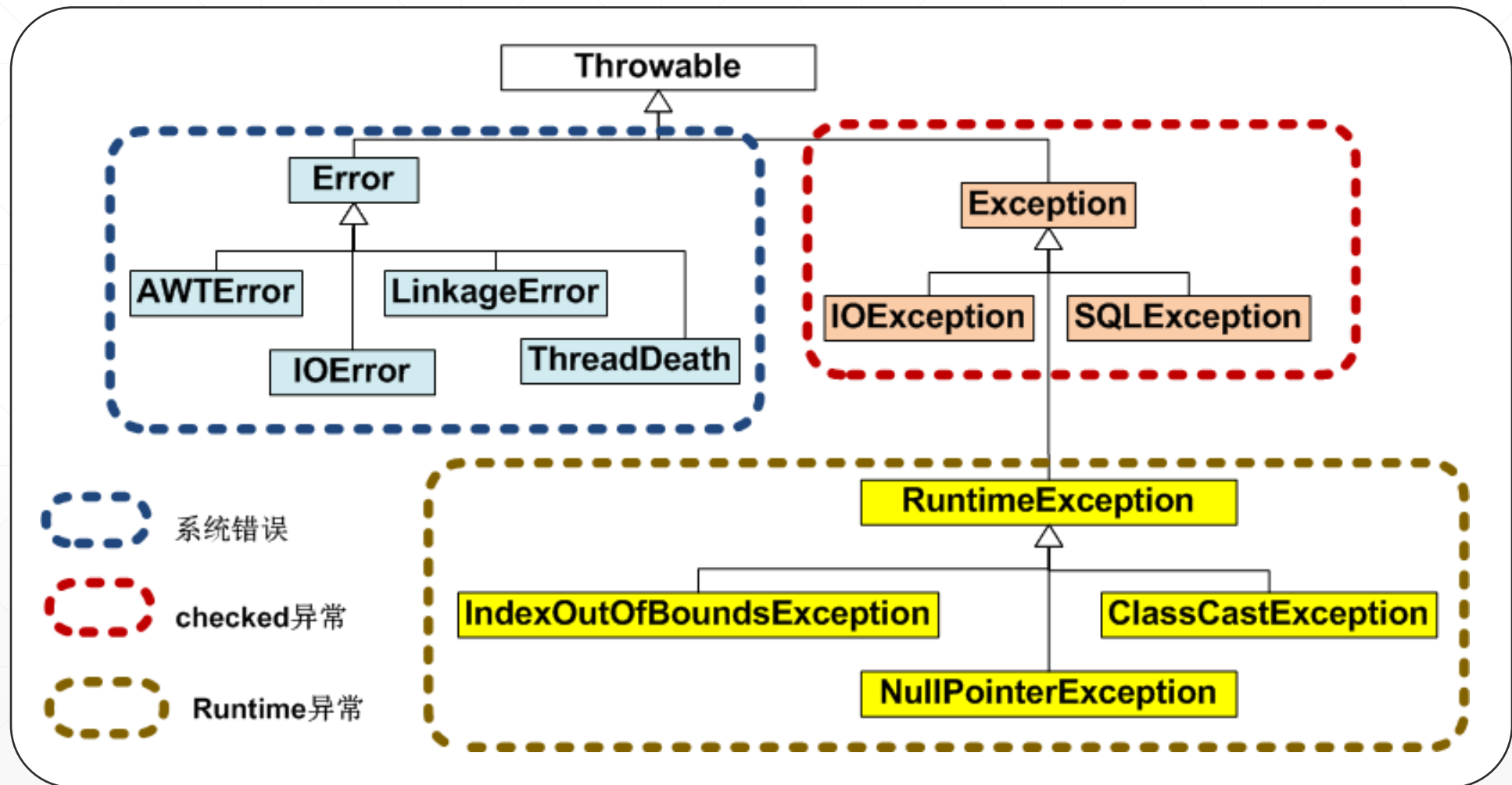
不同类型代码的放置位置

出错处理
代码

```
try {  
    //可能发生运行错误的代码  
}  
catch(异常类型1 异常对象引用) {  
    //用于处理异常1的代码  
}  
catch(异常类型2 异常对象引用) {  
    //用于处理异常2的代码  
}  
finally{  
    //用于“善后”的代码  
}
```

业务逻辑代码

三种类型的“异常”



Java中的异常分类



- Throwable类是最顶层的类，它有两个直接子类：
 1. **Exception**：出现的问题是可以被捕获的；
 2. **Error**：系统错误，通常由JVM处理。
- 可捕获的异常（Exception及其子类）又可以进一步细分为两类：
 1. **check异常（受控的异常）**：直接派生自Exception的异常类，必须被捕获或在方法定义时再次声明抛出
 2. **runtime异常（运行时异常）**：派生自RuntimeException的异常类。使用throw语句可以随时抛出这种异常对象：

```
throw new ArithmeticException(...);
```



受控异常的相关特性，后面课程介绍。

一个著名的Error——OOM Error

- 在早期的Android应用开发中，我们会经常碰到一个Error，就是“**OutOfMemoryError**”，表示系统内存不足，这个Error简称为“**OOM Error**”。
- Android应用在处理大图片等耗费大量内存资源的工作时，就有可能发生OOM。
- 通常情况下，Error出现的可能性很低，并且不可控，所以在Java应用程序中一般不需要专门处理。但在Android应用开发中，捕获OOM往往是必要的，主要目的是为了能手动释放内存，以避免由于占用大量内存影响Android手机的响应速度和App的运行流畅程度。



OOM在早期的Android应用中经常出现，主要原因一在于Android操作系统还不够成熟，二在于Android手机所配置的物理内存通常不大。但近年来，随着手机硬件性能的不不断提升，手机上动辄配置上G的内存，OOM已经出现得不多了。

在必要时，应用程序也可以主动抛出Error

- JDK 1.4 以上提供了assert语句，允许程序在运行期间判断某个条件是否满足，不满足时，抛出AssertionError
- 例如：

TestAssert.java

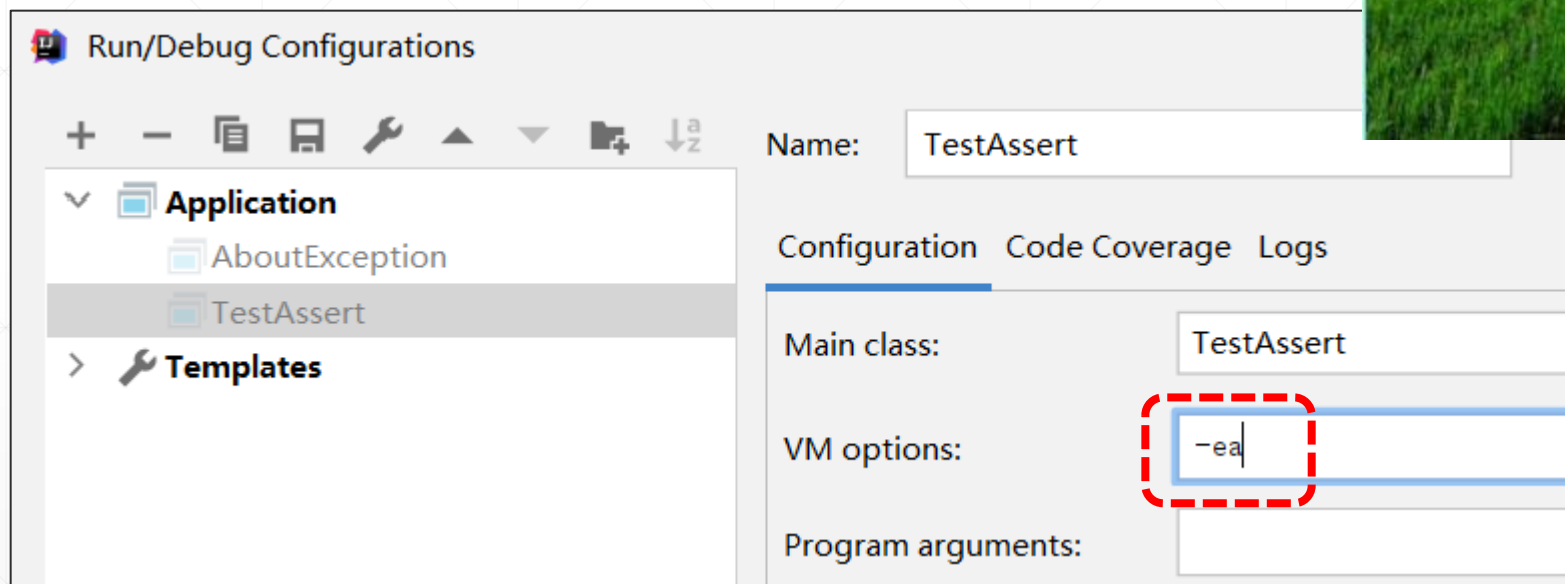
```
public static void main(String[] args) {  
    List<Integer> ints = Arrays.asList(1,2,3);  
    int s = 0;  
    for (int n : ints) { s += n; }  
    assert s == 7;  
}
```

运行结果：

```
Exception in thread "main" java.lang.AssertionError  
    at TestAssert.main(TestAssert.java:9)
```

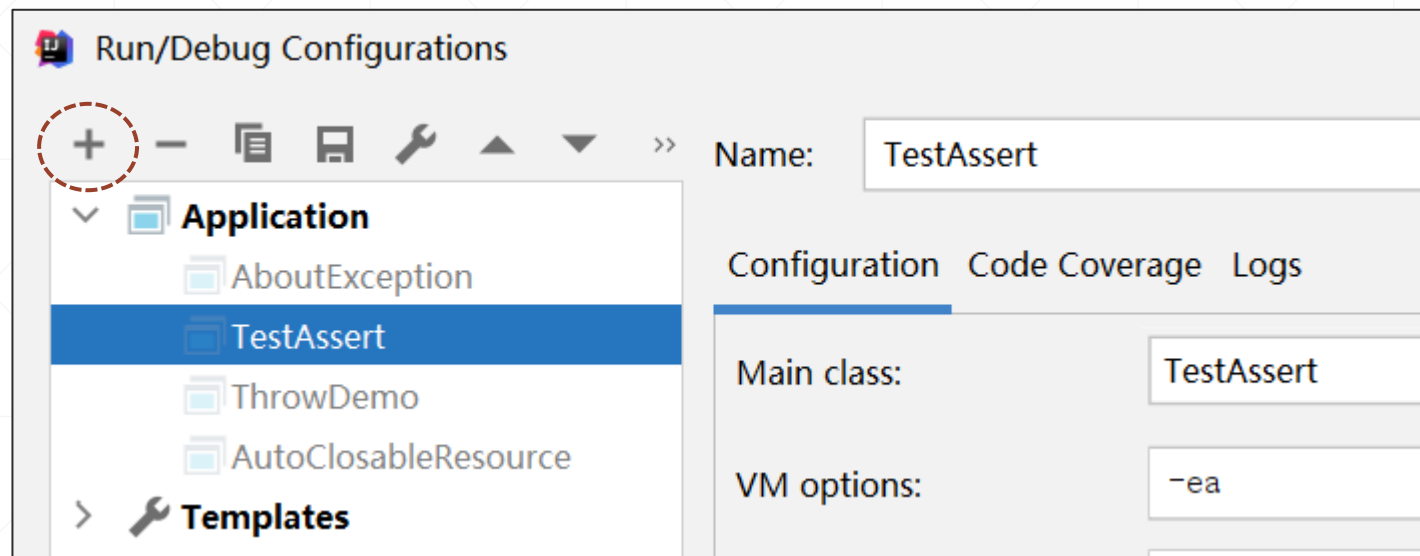
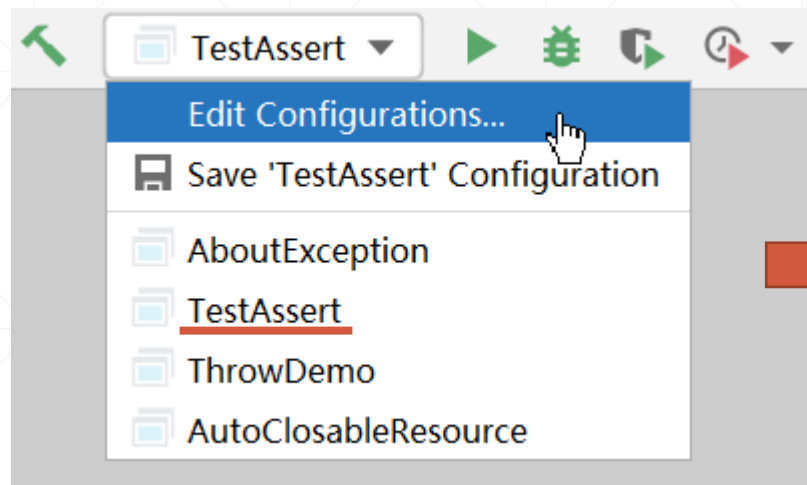
启用assert功能

- 默认情况下，assert功能是关闭的，可以在使用java启动JVM时添加参数 `-ea` 打开它。



使用这一功能，主要用于快速定位与尽早发现可能引发错误的代码。你可以将assert语句视为“代码Bug监测站”。

多了解一点



在一个Java项目中，可以包容多个类，每个类都可以有自己的main方法。IntelliJ允许你为每个类的main方法指定不同的“运行环境”，这一环境的相关参数，在“Run/Debug Configurations”中设置。

可以点击“+”号，从IntelliJ所提供的模板中选一个为特定类的main方法指定运行环境，但最简单的方式是在代码编辑器中直接点击main方法旁的“绿色三角”按钮启动运行，IntelliJ会自动在上左图所示的下拉列表中添加一项，之后你就可以直接修改它所对应的参数了。

请尝试解释以下的“灵异”现象！

```
int i=1, j=0, k;  
k=i/j;
```

左边代码在运行时
会引发异常

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
at AboutException.main(AboutException.java:9)
```

```
double d1 = 100, d2 = 0, result;  
result = d1/d2;  
System.out.println("浮点数除以零: " + result);
```

上边代码在运行时却不会引发
异常，为什么会这样？

浮点数除以零: Infinity

参看实例: ThrowDemo.java

技术探索



前页PPT中所出现之奇怪现象，请你使用javap -c命令去反汇编示例程序生成的.class文件，看看有没有什么发现。

参考解答

```
int i = 1, j = 0, k;  
k = i/j;
```

javac生成



idiv字节码指令

```
double d1 = 100, d2 = 0, result;  
result = d1/d2;  
System.out.println("浮点数除以零: " + result);
```

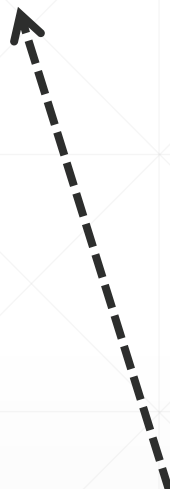


javac生成

ddiv字节码指令



两个除法生成不同的字节码指令，相应地，JDK采用了不同的处理策略，导致两段代码运行时得到不同的结果



继续学习—了解异常的“多态”特性



```
try {  
    //可能发生运行错误的代码;  
}  
catch(异常类型1 异常对象引用 ) {  
    //用于处理异常1的代码  
}  
catch(异常类型2 异常对象引用) {  
    //用于处理异常1的代码  
}  
finally{  
    //用于“善后”的代码  
}
```

- 可以有多个catch语句块，每个代码块捕获一种异常。如果有两个不同的catch块捕获相同类型的异常，这是一个语法错误。
- 当异常发生时，JVM按照catch块的顺序从上往下顺序匹配，找到第一个匹配的catch块时，就停止这个匹配过程，然后执行这个catch块中的代码。
- 由于Exception是所有可捕获异常的基类，它就可以匹配所有异常对象，所以，一个捕获Exception对象的catch语句块可以捕获所有的异常。



进行异常捕获时不仅应该把Exception类对应的catch块放在最后，而且所有父类异常的catch块都应该排在子类异常catch块的后面（即“**先处理小异常，再处理大异常**”），否则将出现编译错误。

动手动脑：多层的异常捕获-1



- 阅读以下两层嵌套的异常捕获示例代码，写出程序运行结果：

```
private static void test1() {  
    try {  
        try {  
            throw new ArrayIndexOutOfBoundsException();  
        }  
        catch(ArrayIndexOutOfBoundsException e) {  
            System.out.println( "ArrayIndexOutOfBoundsException" + "/内层try-catch");  
        }  
        throw new ArithmeticException();  
    }  
    catch(ArithmeticException e) {  
        System.out.println("发生ArithmeticException");  
    }  
    catch(ArrayIndexOutOfBoundsException e) {  
        System.out.println( "ArrayIndexOutOfBoundsException" + "/外层try-catch");  
    }  
}
```

CatchWho.java

动手动脑：多层的异常捕获-2



- 阅读以下代码，写出程序运行结果：

CatchWho.java

```
private static void test2(){
    try {
        try {
            throw new ArrayIndexOutOfBoundsException();
        }
        catch(ArithmeticException e) {
            System.out.println( "ArrayIndexOutOfBoundsException" + "/内层try-catch");
        }
        throw new ArithmeticException();
    }
    catch(ArithmeticException e) {
        System.out.println("发生ArithmeticException");
    }
    catch(ArrayIndexOutOfBoundsException e) {
        System.out.println( "ArrayIndexOutOfBoundsException" + "/外层try-catch");
    }
}
```

示例中的test1和test2方法高度类似，但输出结果不一样，请试着总结规律。

“finally”块的功用

- **资源泄露**：当一个资源不再被某应用程序使用，但此程序并未向系统声明不再使用此资源时发生这种情况（即资源虽然空闲但却无法再被使用）
- **finally**语句块主要用于解决资源泄露问题，它位于**catch**语句块之后，可以在里面写代码释放程序所占用的资源，JVM会在各种异常被抛出时，保证**finally**语句块都能得以执行。
- **finally**语句块，类似于高压锅锅盖上的泄压阀——泄压阀在高压锅内蒸汽压力过大时，会自动打开放气，以避免引发危险。

```
try {  
    //可能发生运行错误的代码;  
}  
catch(异常类型1 异常对象引用 ) {  
    //用于处理异常1的代码  
}  
catch(异常类型2 异常对象引用) {  
    //用于处理异常2的代码  
}  
finally {  
    //用于“善后”的代码  
}
```



下一讲将会讨论系统资源的释放问题。

动手动脑



- 当有多个嵌套的try...catch...finally时，要特别注意finally的执行时机。
- 本讲所配的EmbedFinally.java示例包容一个多层嵌套的try/catch/finally结构，请先阅读它，尝试着推断出程序的运行结果，再运行它，观察其真实输出并进行总结。

- 特别注意：

当有多层嵌套的finally语句块时，异常在不同的层次抛出，在不同的位置抛出，可能会导致不同的finally语句块执行顺序。

一个思考题：



- 辨析：在任何情况下，finally语句块都一定会执行吗？
- 请通过 SystemExitAndFinally.java 示例程序回答上述问题。

```
public class SystemExitAndFinally {  
    public static void main(String[] args) {  
        try {  
            System.out.println("in main");  
            //System.exit(0);  
            throw new Exception("Exception is thrown in main");  
        } catch (Exception e) {  
            System.out.println(e.getMessage());  
            //System.exit(0);  
        } finally {  
            System.out.println("in finally");  
        }  
    }  
}
```

试着取消System.exit()语句的注释，看看结果是什么！

参考解答



如果在异常处理代码中使用 `System.exit()` 语句直接退出，则 `finally` 块将失去执行的机会！

课后练习：自我归纳与总结



- 依据对本讲多个示例程序的分析，请自行归纳总结出Java多层嵌套异常处理的基本流程。
- 阅读任何一本Java教材，或者是使用互联网搜索引擎，查找有关Java多层嵌套异常处理流程的资料，看看你总结得是否全面。

只有自己总结出来的知识才是自己的！

如何跟踪异常的传播路径？

- 当程序运行过程中抛出了异常对象时，JVM会依据抛出异常的那个方法调用的顺序依次查找关联的异常处理程序。
- 可使用`printStackTrace` 和 `getMessage`方法了解异常发生的情况：
 1. `printStackTrace`：打印方法调用异常跟踪栈。
 2. 每个`Throwable`类的对象都有一个`getMessage`方法，它返回一个字符串，这个字符串是在`Exception`构造函数中传入的，通常让这一字符串包含特定异常的相关信息

打印“异常跟踪栈”信息

```
public static void method1() throws Exception {  
    method2();  
}  
public static void method2() throws Exception {  
    method3();  
}  
public static void method3() throws Exception {  
    throw new Exception("在method3中抛出了异常");  
}
```

```
public static void main(String[] args) {  
    try {  
        method1();  
    } catch (Exception e) {  
        System.err.println(e.getMessage());  
        System.err.println("以下是打印出来的异常堆栈信息.....");  
        e.printStackTrace();  
    }  
}
```

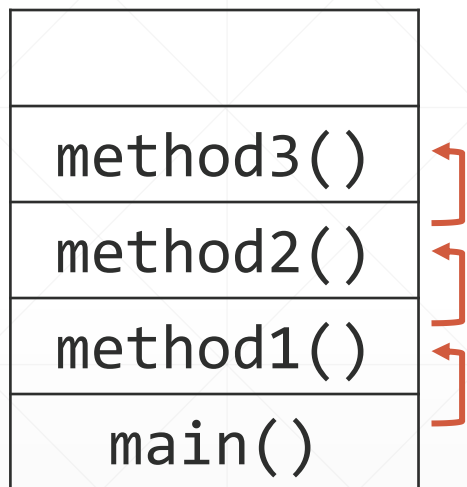
PrintExceptionStack.java

示例运行结果:

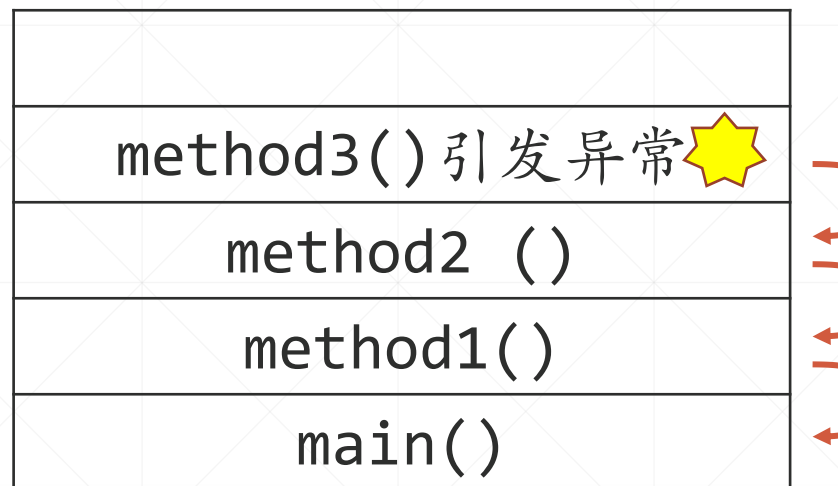
```
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent:C:\Program  
在method3中抛出了异常  
以下是打印出来的异常堆栈信息.....  
java.lang.Exception Create breakpoint : 在method3中抛出了异常  
    at PrintExceptionStack.method3(PrintExceptionStack.java:21)  
    at PrintExceptionStack.method2(PrintExceptionStack.java:17)  
    at PrintExceptionStack.method1(PrintExceptionStack.java:13)  
    at PrintExceptionStack.main(PrintExceptionStack.java:4)
```

两个堆栈

方法调用堆栈



异常对象传递堆栈
(跟踪栈)



"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent:C:\Program
在method3中抛出了异常
以下是打印出来的异常堆栈信息.....
java.lang.Exception Create breakpoint : 在method3中抛出了异常
at PrintExceptionStack.method3(PrintExceptionStack.java:21)
at PrintExceptionStack.method2(PrintExceptionStack.java:17)
at PrintExceptionStack.method1(PrintExceptionStack.java:13)
at PrintExceptionStack.main(PrintExceptionStack.java:4)

小结



本讲介绍了Java程序中异常处理的基础知识与基本语法特性，非常重要，但这些仅仅只是Java异常处理机制特性的一部分。

下一讲，系统介绍Java中受控异常及其相关的语法特性。